The Shortest-Path Problem

A correct procedure on the wrong model is still wrong

COMSC 2043 • Discrete Structures and Critical Thinking

Week 12, Thursday, November 5, 2026

How to read these slides

Orange tag, top right

PRINCIPLE: the idea of the course that this slide shows.

- MODEL VS PROCEDURE
- ASYMMETRY
- COUNTING / PATTERN
- GREEDY IS NOT OPTIMAL
- CHECK THE WORK

Red box: PROMPT FOR CLAUDE

Paste-ready text for Claude, only on the slides that need code.

What you should see: the real numbers, computed by the committed code.
PREDICT (30 s): say your guess out loud first.

Today's path

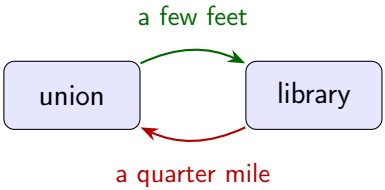
1. A one-way street (the hook)
2. A six-sided die gives six cities, and a table of distances
3. A tour: 1-2-3-4-5-6-1, and what we are optimizing
4. Greedy: always go to the nearest city, and where it is fooled
5. Guess and check with the die
6. How many different tours? A pattern, and a plot
7. Then we started writing code
8. Close: model versus procedure, and the Gate

Words we will use

graph	Places (dots) and links between them (arrows or lines).
directed edge	A link with a direction: it goes from one place to another.
weight	A number on a link, here a distance.
tour	A trip that visits every city once and comes back to the start.
greedy	Always take the cheapest-looking next step.
brute force	Try every possibility and keep the best.
factorial ($k!$)	$k \times (k-1) \times \cdots \times 1$. For example $4! = 4 \cdot 3 \cdot 2 \cdot 1 = 24$.
permutation	One ordering of a list of things.
test (assert)	A line of code that stops with an error if a claim is false.

Union to library

- Student union to the library: a few feet. Walk across.
- Library back to the union: a quarter mile of driving, because the street is one way.



Same two places. Two different distances.

PREDICT (30 s): Is the distance from A to B always the distance from B to A?

Give it a name: a directed, weighted graph

- Each place is a **vertex** (a dot).
- Each one-way trip is a **directed edge** (an arrow), with its own **weight** (the distance).
- Union→library and library→union are *two different edges* with two different weights.

This is called an **asymmetric distance**.

The model says

The edge has a direction and its own weight.

If we got the model wrong

A single number for "union–library" is wrong in one direction, whatever we compute with it.

A six-sided die gives six cities

PRINCIPLE: ASYMMETRY

Six cities, numbered 1 to 6. Row = where you start, column = where you go. The shaded diagonal is blank: no trip from a city to itself.

from \ to	1	2	3	4	5	6
1		11	4	12	5	6
2	9		12	10	10	4
3	12	11		10	7	12
4	10	9	9		6	12
5	12	3	6	8		6
6	10	9	2	11	13	

What you should see: `1 -> 2` costs 11 but `2 -> 1` costs 9. Every pair differs on purpose.

Reading the table

from \ to	1	2	3	4	5	6
1		11	4	12	5	6
2	9		12	10	10	4
3	12	11		10	7	12
4	10	9	9		6	12
5	12	3	6	8		6
6	10	9	2	11	13	

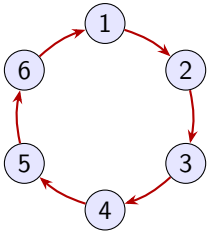
- Find the row for where you are.
- Slide right to the column for where you want to go.
- That cell is one leg of a trip.

PREDICT (30 s): What does it cost to go from 4 to 2? From 2 to 4?
What you should see: 4 to 2 costs 9; 2 to 4 costs 10.

A tour, then home

Try the simplest order: **1, 2, 3, 4, 5, 6, and back to 1**.

- Six cities means six legs, because the trip ends where it started.
- Each leg is one cell of the table: read it *from* the row *to* the column.



PREDICT (30 s): How many legs does the tour have? (Careful: not five.)

Total the six legs, step by step

PRINCIPLE: CHECK THE WORK

from \ to	1	2	3	4	5	6
1		**11**	4	12	5	6
2	9		**12**	10	10	4
3	12	11		**10**	7	12
4	10	9	9		**6**	12
5	12	3	6	8		**6**
6	**10**	9	2	11	13	

$1 \to 2$: 11 $\quad 2 \to 3$: 12 $\quad 3 \to 4$: 10

$4 \to 5$: 6 $\quad 5 \to 6$: 6 $\quad 6 \to 1$: 10

What you should see: $11 + 12 + 10 + 6 + 6 + 10 = \mathbf{55}$

What are we optimizing?

The objective

Find the **closed tour** that visits **every city exactly once** and has the **smallest total length**.

- Our first tour, 1-2-3-4-5-6-1, has total **55**. Can we do better?
- "Shortest" means nothing until we say *what* is measured (total length), *over what* (closed tours), and *in which direction* (the table).

What you should see: one tour, total 55. That is a number to beat, not an answer.

Idea 1: greedy, always go to the nearest city

PRINCIPLE: GREEDY IS NOT OPTIMAL

Rule: from where you are, look at the cities you have not visited and go to the one with the smallest distance. Ties: lowest city number.

Start at city 1.

- From 1: options 2 (11), 3 (4), 4, 5 (5), 6. Nearest is 3 at 4.

PREDICT (30 s): From 3, which unvisited city is nearest?

Greedy, worked on the table from city 1

PRINCIPLE: GREEDY IS NOT OPTIMAL

from \ to	1	2	3	4	5	6
1		11	**4**	12	5	6
2	9		12	10	10	**4**
3	12	11		10	**7**	12
4	**10**	9	9		6	12
5	12	**3**	6	8		6
6	10	9	2	**11**	13	

$1 \to 3$	4
$3 \to 5$	7
$5 \to 2$	3
$2 \to 6$	4
$6 \to 4$	11 (only 4 is left)
$4 \to 1$	10 (the way home)

What you should see: tour **1-3-5-2-6-4-1**, total **39**. Better than 55.

Where greedy is fooled

Greedy

1-3-5-2-6-4-1, total 39

A better tour (found by trying all of them)

1-5-2-6-3-4-1, total 34

- Greedy's first step took 1→3 (4) instead of 1→5 (5): it saved 1.
- But with 3 already visited, the cheap leg 6→3 (2) could never be used. Greedy ended with 6→4 at 11.

What you should see: greedy loses by 5: 39 versus 34. The "best" tour is found later in the deck by code.

Idea 2: guess and check with the die

PRINCIPLE: CHECK THE WORK

Start at city 1. Roll the die for the next city.

- If the city is already in the tour, **reject** the roll and roll again.
- When all six cities are in, add the way home, total the six legs, and write it down.
- Keep the **best so far**. Repeat.

PREDICT (30 s): Is a rejected roll free?

What rejecting repeats costs

PRINCIPLE: COUNTING / PATTERN

- Slot 2: five of six faces work. Slot 3: four of six. . . . Slot 6: only one face works.
- With k cities still open out of 6, you need $6/k$ rolls on average.
- Over five slots: $6/5 + 6/4 + 6/3 + 6/2 + 6/1$.

What you should see: about **13.7** rolls to fill one tour, not 5.

A short worked trace

#	rolls for slots 2–6	tour	total	best so far
1	2 3 5 4 4 6	1-2-3-5-4-6-1	60	60
2	3 4 3 2 4 5 2 2 6	1-3-4-2-5-6-1	49	49
3	5 2 5 6 5 4 3	1-5-2-6-4-3-1	44	44
4	6 6 5 2 4 3	1-6-5-2-4-3-1	53	44
5	2 4 2 5 3 1 3 2 2 6	1-2-4-5-3-6-1	55	44

What you should see: best after 1 guess: 60. After 3: 44. After 5: 44. It only ever goes down or stays.

Guessing cannot know it is done

PRINCIPLE: CHECK THE WORK

- Our best tour is 34. After five guesses our best is 44, not 34, and nothing tells us a better tour exists.
- Guess 1000 times and the best so far reaches 34.
- That is more guesses than there are tours. With 6 cities, the count of tours is only 120.

What you should see: best after 1000 guesses = 34, matching the full search later.

Count the tours: 2 cities, then 3

PRINCIPLE: COUNTING / PATTERN

Fix the start at city 1. Count the closed tours that visit every city once.

2 cities: 1-2-1 $\Rightarrow$ **1** tour.

3 cities: 1-2-3-1 1-3-2-1 $\Rightarrow$ **2** tours.

PREDICT (30 s): How many tours for 4 cities? Write down a guess before the next slide.

4 cities

All the orders of cities 2, 3, 4 after the start:

1-2-3-4-1 1-2-4-3-1 1-3-2-4-1 1-3-4-2-1 1-4-2-3-1 1-4-3-2-1

What you should see: **6** tours.

- 3 choices for the first stop, then 2 for the second, then 1 for the last.

Choices shrink by one each step

PRINCIPLE: COUNTING / PATTERN

For n cities with the start fixed, there are $n-1$ cities left to order:

slot	2nd stop	3rd	4th	5th	6th
choices (6 cities)	5	4	3	2	1

Multiply: $5 \times 4 \times 3 \times 2 \times 1 = 5! \quad \Rightarrow \quad (n-1)!$ tours.

What you should see: 24 tours for 5 cities, 120 tours for 6.

The pattern

cities n	2	3	4	5	6
tours $(n-1)!$	1	2	6	24	120

PREDICT (30 s): What is the next number, for 7 cities?

- Each step multiplies by the next whole number: $\times 2$, $\times 3$, $\times 4$, $\times 5$.

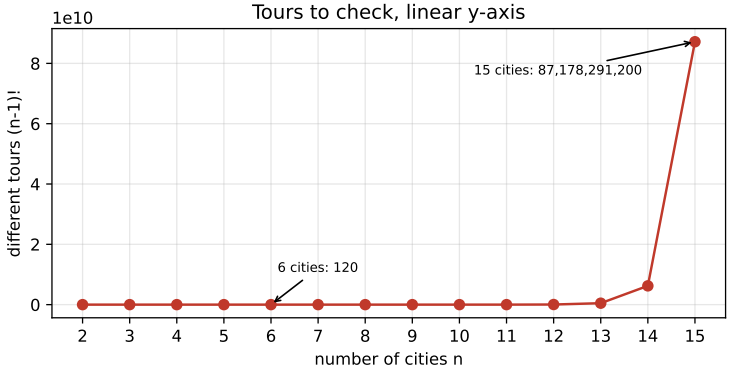
Which counting convention? Say it once

PRINCIPLE: COUNTING / PATTERN

- **Our convention:** start fixed at city 1, direction matters. Count $= (n-1)!$.
- Do not fix the start: list all $n!$ orders, but each tour appears n times (rotations), and you get the same $(n-1)!$ tours. For 6 cities: 720 listings, 120 tours.
- Ignore direction (a two-way street): a tour and its reverse are the same, so halve: $(n-1)!/2$, which is 60 for 6 cities.

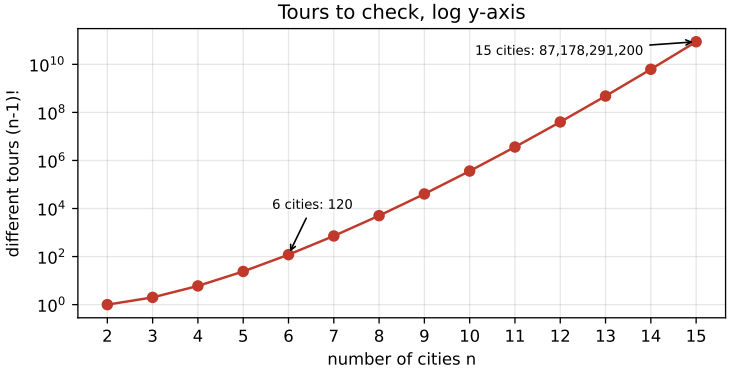
What you should see: Our table is asymmetric, so a tour and its reverse are different tours. **No halving.**

Plot it (linear)



What you should see: the curve is flat for a few cities, then it shoots off the page.

Plot it (log)



What you should see: each step up the axis is a factor of ten; the line still bends upward.

What the curve means for brute force

- 6 cities: **120** tours. A laptop does that instantly.
- 10 cities: 362 880 tours.
- 15 cities: **87 178 291 200** tours (about 87 billion).

What you should see: at one million tours per second, 15 cities takes about 24.2 hours. (The speed is an assumption.)

Then we started writing code

We now know the model (a directed, weighted table) and the objective (the shortest closed tour). Only now do we code it.

1. read the table
2. `tour_length(tour)`
3. check 1-2-3-4-5-6-1 against our hand total
4. greedy
5. guess and check with `random`
6. brute force with `itertools.permutations`, and check the count

Step 1: read the table

PROMPT FOR CLAUDE

prompts/01_read_table.md

```
Make a folder called tour_demo. Put data/distances.csv in it (I will paste it).
Write tour.py with a function read_table(path) that reads the CSV and returns a dictionary
{(from_city, to_city): distance} using whole numbers. The diagonal cells are blank, so skip them.
Do not add anything else yet.
Then add a few lines at the bottom (under an if __name__ == "__main__": block) that print
the distance from 1 to 2 and from 2 to 1. Run it and show me the output.
```

What you should see: 11 for 1 to 2, and 9 for 2 to 1: the two directions differ.

PREDICT (30 s): Will the two printed numbers be equal?

Step 2: the length of a tour

PRINCIPLE: CHECK THE WORK

PROMPT FOR CLAUDE

prompts/02_tour_length.md

```
In tour.py add tour_length(tour, dist). tour is a list of city numbers like [1, 2, 3, 4, 5, 6].
It must add up every leg, INCLUDING the leg from the last city back to the first.
Add a second function legs(tour, dist) that returns each leg as (from_city, to_city, distance)
so I can compare it with my hand work. Keep both functions short. Do not change read_table.
```

What you should see: `legs` gives six triples; `tour_length` adds their distances.

PREDICT (30 s): What goes wrong if the code forgets the leg from the last city back to the first?

Step 3: check our hand total

PRINCIPLE: CHECK THE WORK

PROMPT FOR CLAUDE

prompts/03_check_tour.md

```
In tour_demo, run legs([1, 2, 3, 4, 5, 6], dist) and tour_length on the same tour.
Print the six legs and the total. I will compare them with the legs and total I added up by hand.
If they differ, do not change my numbers: tell me which leg differs and why.
```

What you should see: legs 11, 12, 10, 6, 6, 10; total **55**, the same as our hand total.

Step 3b: tests guard the model

PRINCIPLE: ASYMMETRY

PROMPT FOR CLAUDE

prompts/03_check_tour.md

```
Create test_tour.py with pytest tests using read_table("data/distances.csv"):
1. the leg 1 to 2 and the leg 2 to 1 are NOT equal (the table is asymmetric);
2. tour_length([1, 2, 3, 4, 5, 6]) equals the total I give you (I will paste my hand total).
Run pytest and show me the one-line summary.
```

What you should see: `2 passed`. Test 1 fails if someone "fixes" the table to be symmetric.

Step 4: greedy

PROMPT FOR CLAUDE

prompts/04_greedy.md

```
In tour.py add greedy_tour(dist, start=1). From the current city, go to the unvisited city
with the smallest distance. If two cities tie, take the lower city number. Return the tour as a list.
Run it from city 1, print the tour and tour_length. Show me each step: current city, the options
with their distances, and the one it chose, so I can check it against my hand work.
```

What you should see: tour 1-3-5-2-6-4-1, total **39**, matching the hand work.

PREDICT (30 s): Which city does greedy choose from city 6?

Step 5: guess and check with random

PRINCIPLE: CHECK THE WORK

PROMPT FOR CLAUDE

prompts/05_guess_and_check.md

```
In tour.py add guess_tour(dist, rng, start=1): keep the start city, then pick a random city from
all six with rng.choice(); if it is already in the tour, reject it and pick again; return the tour
AND the list of every roll (rejected ones too).
Add guess_and_check(dist, k, seed): make k tours with random.Random(seed), and for each print
the rolls, the tour, its total, and the best total so far. Run it with k=5 and seed=3114 and show me the output.
```

What you should see: with seed 3114: best after 1 guess 60, after 3 44, after 5 44; same as the trace slide.

Step 6: brute force over permutations

PRINCIPLE: COUNTING / PATTERN

PROMPT FOR CLAUDE

prompts/06_brute_force.md

```
In tour.py add brute_force(dist, start=1) using itertools.permutations on the other cities, so the
start city is fixed. For each order build the tour, compute tour_length, and keep the best.
Return (best_tour, best_total, count_of_tours_tried). Run it and show me all three values.
```

What you should see: best tour 1-5-2-6-3-4-1, total **34**, 120 tours tried.

PREDICT (30 s): Will brute force beat greedy, tie it, or lose to it?

Step 6b: assert the count

PRINCIPLE: COUNTING / PATTERN

PROMPT FOR CLAUDE

prompts/06_brute_force.md

```
Add two pytest tests to test_tour.py:
1. for n = 2 to 6 (use only the first n cities of the table), the count brute_force returns equals
   math.factorial(n - 1);
2. brute_force's best total is less than or equal to the total of greedy_tour.
Run pytest and show me the one-line summary.
```

What you should see: `pytest` passes: counts 1, 2, 6, 24, 120 for 2 to 6 cities, and brute force is no worse than greedy.

Model versus procedure: each step, again

Step	What it assumed	What would change
the table	the street is one way: each direction has its own weight	two-way: one number per pair, tours and reverses tie
the tour	visit every city once and come home	drop "come home": an open path, a different problem
greedy	the cheapest next step belongs to the cheapest tour	nothing in the table promises that
guess and check	random tours sample the space well	sample is tiny when the count explodes
brute force	the count is small enough to try all	fine at 6 cities, hopeless at 15

If the street were two-way

- One number per pair of cities, not two. The table would mirror across its diagonal.
- A tour and its reverse would have the same total, so the tour count halves.
- Our greedy and brute-force code would still run, and still produce an answer. It would be the answer for the wrong street.

The course's own question

Are we solving the graph correctly, or modeling reality incorrectly?

Where this came from

Jeremy first worked this as the shortest path problem in Python in 2020 to 2021: brute force, greedy, then ant colony ideas and MPI on a cluster.

Gate preview

```
# Week 12 Decision Gate - Graph model, claim, and boundary

This individual artifact is the Week 12 ordinary **Decision Gate** in the 30%
category, not a second copy of the Show & Tell report.

Choose a bounded network question from class or an instructor-approved,
non-sensitive source. A diagram, table, or program is acceptable.

1. **Sources** - facts/data, distinguished from graph choices.
2. **Rules / Assumptions** - vertices, edges, direction, weights, and
   objective.
3. **Work** - graph plus reproducible path/traversal/connectedness reasoning.
4. **Check** - challenge a modeling assumption, verify path steps, or compare
   another method; say whether it tests procedure, model, or both.
5. **One-Sentence Summary** - conclusion plus its model limit.

Label AI suggestions as suggestions and independently verify graph facts,
edge meanings, and calculations. Use ‘assignments/weekly-problem-solving-writeup.md‘.
```

Exit trace

Before you leave (from the Week 12 Tuesday activity)

State one bounded conclusion and its edge assumption; keep this trace for the Decision Gate.

- Bounded: true for this table and this objective, not for every street.
- Edge assumption: what does one arrow mean, and does it have a direction?

Index 1 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
5	Union to library	ASYMMETRY	
6	Give it a name: a directed, weighted graph	MODEL VS PROCEDURE	
7	A six-sided die gives six cities	ASYMMETRY	
9	A tour, then home	MODEL VS PROCEDURE	
10	Total the six legs, step by step	CHECK THE WORK	
11	What are we optimizing?	MODEL VS PROCEDURE	
12	Idea 1: greedy, always go to the nearest city	GREEDY IS NOT OPTIMAL	
13	Greedy, worked on the table from city 1	GREEDY IS NOT OPTIMAL	
14	Where greedy is fooled	GREEDY IS NOT OPTIMAL	

Index 2 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
15	Idea 2: guess and check with the die	CHECK THE WORK	
16	What rejecting repeats costs	COUNTING / PATTERN	
17	A short worked trace	CHECK THE WORK	
18	Guessing cannot know it is done	CHECK THE WORK	
19	Count the tours: 2 cities, then 3	COUNTING / PATTERN	
20	4 cities	COUNTING / PATTERN	
21	Choices shrink by one each step	COUNTING / PATTERN	
22	The pattern	COUNTING / PATTERN	
23	Which counting convention? Say it once	COUNTING / PATTERN	

Index 3 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
24	Plot it (linear)	COUNTING / PATTERN	
25	Plot it (log)	COUNTING / PATTERN	
26	What the curve means for brute force	COUNTING / PATTERN	
27	Then we started writing code	MODEL VS PROCEDURE	
28	Step 1: read the table	MODEL VS PROCEDURE	01a
29	Step 2: the length of a tour	CHECK THE WORK	02a
30	Step 3: check our hand total	CHECK THE WORK	03a
31	Step 3b: tests guard the model	ASYMMETRY	03b
32	Step 4: greedy	GREEDY IS NOT OPTIMAL	04a

Index 4 of 4: principle, slide, prompt

Slide	Title	Principle	Prompt
33	Step 5: guess and check with random	CHECK THE WORK	05a
34	Step 6: brute force over permutations	COUNTING / PATTERN	06a
35	Step 6b: assert the count	COUNTING / PATTERN	06b
36	Model versus procedure: each step, again	MODEL VS PROCEDURE	
37	If the street were two-way	MODEL VS PROCEDURE	
39	Gate preview	CHECK THE WORK	
40	Exit trace	CHECK THE WORK	